

APPSEC-05: Security Posture Management

Series: APPSEC — Application Security | **Notebook:** 5 of 10 | **Created:** June 2026 |
Last Updated: 08/25/2026

Overview

Security Posture Management (SPM) evaluates the configuration state of monitored entities — Kubernetes clusters, cloud accounts, container images — against compliance baselines (CIS, PCI, NIST) and surfaces misconfigurations as findings in the same `security.events` Grail bucket as RVA and RAP.

Where RVA and RAP are *runtime* signals (what's executing right now), SPM is a *config* signal (what's been declared in the platform). The two complement each other — a fully-patched runtime on a misconfigured platform is still vulnerable.



Table of Contents

- 1. [1. SPM Scope](#)
 - 2. [2. Compliance Frameworks](#)
 - 3. [3. DQL: SPM Findings](#)
 - 4. [4. Finding Lifecycle](#)
 - 5. [5. Next Steps](#)
 - 6. [References](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	Gen3 SaaS with Grail; AppSec entitlement enabled
OneAgent	Full-Stack mode (or code-module attached) on monitored hosts
Read access	At minimum <code>environment:roles:view-security-problems</code> and <code>storage:security.events:read</code> — see APPSEC-09 for the full model
Background	APPSEC-01 (fundamentals + three-pillar framing)

1. SPM Scope

SPM's scope is broader than the application — it covers the platform context in which applications run. Typical evaluation targets:

Entity class	What SPM checks
Kubernetes clusters	RBAC, pod security, network policies, secret management
Cloud accounts (AWS / Azure / GCP)	IAM permissions, logging, storage encryption, public-exposure config
Container images	Base-image age, known vulnerabilities, secret scanning

Entity class	What SPM checks
Dynatrace platform itself	IAM hygiene, ingestion auth scheme (see AUTOM-04 § 3)

The exact catalog of checks lives in the SPM product surface and evolves per release.

Sources: [Application Security \(DT docs\)](#) for SPM as the third pillar. **Softened:** the per-entity-class check catalog is community-practice synthesis; the deep-page SPM docs were not resolvable at 06/04/2026 to enumerate verbatim.

2. Compliance Frameworks

SPM findings can be grouped by framework so a single misconfiguration shows up under each relevant standard:

- **CIS Benchmarks** — Center for Internet Security baselines per platform
- **PCI DSS** — payment card industry data security
- **NIST 800-53** — US federal control catalog
- **HIPAA / SOC 2 / ISO 27001** — coverage varies; verify per release

A single finding (e.g., "S3 bucket without encryption") often maps to controls in multiple frameworks. Don't double-count by framework when measuring posture; pick a primary framework for governance reporting and let the others be a secondary view.

Sources: [Application Security \(DT docs\)](#) confirms SPM covers compliance violations. **Softened:** per-framework coverage status should be re-verified per release; the docs page for the full framework catalog was not resolvable at 06/04/2026.

3. DQL: SPM Findings

Filter security events to SPM findings only. As with code-level findings, the exact event-type literal depends on your tenant's vocabulary — run `summarize count() by: {event.type}` first to discover.

```
// SPM compliance findings by standard and severity, last 7 days
fetch security.events, from:-7d
| filter event.type == "COMPLIANCE_FINDING"
| summarize count = count(), by:{compliance.standard.short_name,
compliance.rule.severity.level}
| sort count desc
```

Sources: [Compliance \(Semantic Dictionary\) \(DT docs\)](#). **Live-verified 08/25/2026** against a tenant with SPM enabled: the query above returns CIS findings grouped by severity (MEDIUM 83,077 · CRITICAL 67,675 · HIGH 22,750 · LOW 4,894 across 178,396 findings). An earlier revision of this entry filtered `event.type == "POSTURE_FINDING"` and grouped by `compliance.framework / finding.severity`. **All three identifiers were wrong** — `POSTURE_FINDING` returns nothing over 90 days and is not a documented event type; `compliance.framework` and `finding.severity` are populated on **0** of 178,396 findings. The corrected names are `COMPLIANCE_FINDING`, `compliance.standard.short_name` and `compliance.rule.severity.level`. This is the failure mode SPM queries are most prone to: a wrong identifier is valid DQL, executes cleanly, and returns an empty result that looks exactly like a compliant environment.

4. Finding Lifecycle

SPM findings move through the same lifecycle as RVA findings: OPEN → RESOLVED (config fixed) or OPEN → MUTED / EXCEPTION. Two SPM-specific notes:

1. **Resolution is often automatic** — fix the config and the next SPM scan closes the finding. Manual acknowledgement is more common in RVA than SPM.
2. **Exceptions are common in SPM** — many compliance findings reflect legitimate architectural choices (a public-facing bucket holding intentionally-public assets). Exception with a documented reason is a normal posture.

Sources: [Application Security \(DT docs\)](#) for the SPM framing. **Derived:** the auto-resolution + exception-is-normal observations are synthesis of common SPM operating patterns.

5. Next Steps

1. Run the discovery query (`summarize count() by:{event.type}`) to find your tenant's SPM event-type literals.
2. Pick a primary framework for governance reporting; use others as secondary views.
3. Read **APPSEC-06** for Kubernetes + container security overlap with SPM.
4. Read **APPSEC-10** for SPM dashboard patterns and the executive view.

References

Source	Coverage
Application Security (DT docs)	SPM scope reference

 **DISCLAIMER:** This information was AI generated and is provided "as-is" without warranty. It was produced as an independent, community-driven project and **not supported by Dynatrace**. Always refer to official [Dynatrace documentation](#) for the most current information.